Faculty of Computers and Artificial Intelligence
Cairo University

CS213 - Assignment 2

Course Instructor : Dr. Mohammad El - Ramly

Date : 27 oct 2026

NAME	Reem Hamdy Tawfik Alrefaay (Team leader)	Khaled Mohamed Hussien	Khaled Mohamed Fawzy
ID	20250823	20250212	20250213
Email	20250823@stud.fci-cu.edu.eg	20250212@stud.fci-cu.edu.eg	20250213@stud.fci-cu.edu.eg

Team leader phone: 01551978995

Work breakdown table

20250823 - File designing + Gemini 3.7 Flash conversation & design + Initial design + Memory, RegisterFile & Screen classes

20250212 - Claude Sonnet 5.5 & Gemini 3.8 Flash (Antigravity) designs + Instruction, ALU & ProgramLoader C++ classes

20250213 - Comparing the 3 AI designs, Final Design (Gemini Argon 4), Examples, Explanation Video, CPU, Machine, UI & GUI

Introduction :

The Vole Machine architecture consists of three core hardware components:

1. Main Memory: 256 addressable cells, each cell holding an 8-bit value which is either an 8-bit instruction or data value represented as a 2-byte hex value (2 hex digits per byte).

2. Register File: 16 general purpose registers (numbered 0 to F) each of which can hold an 8-bit value. These are the registers used to store data as it's being manipulated during the execution of a program.

3. Central Processing Unit (CPU): Responsible for executing the instruction cycle, which includes:

Fetching: the instruction is retrieved from memory via the Program Counter (PC).

Decoding: interpreting the 4 character hex instruction to work out what type of operation it is and what the operands are.

Executing: carrying out arithmetic, logical or data transfer operations (e.g. LOAD, ADD, STORE, JUMP).

Project Objectives:

The main objective of this project is to apply OOP in C++ to model the various architectural elements in a clean and modular fashion. We document the design process throughout the report, from the initial conceptual design through to the evaluation of various alternative architectures produced by AI models, the selection of the best final architecture and the actual implementation of sample Vole machine programs.

Initial Design:

1.Overview :
we apply OOP principles to separate responsibilities across machine components , ensuring clean maintenance and data management

2.Classes and responsibilities :
Memory :
simulates main memory (256 cells) using a vector ,with methods like read(),write()and clear().
Registers :
manage 16 general-purpose registers (0-F),pc and IR with getRegister() and setRegister().
CPU:
execute the fetch -decode-execute cycle via fetch().decode() and execute().
volesimulator:
controls overall system flow ,user menusand program execution (loadProgram() , run(),step(), displayStatus().)

3.Relationships:
compostion between volesimulator and core components ;association between CPU,MEMORY and REGISTERS

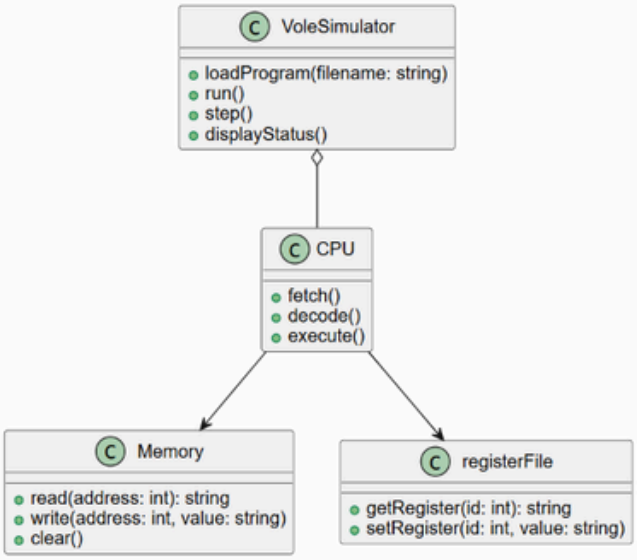
```
class Memory {
+read(address: int): string
+write(address: int, value: string)
+clear()
}

class registerFile {
+getRegister(id: int): string
+setRegister(id: int, value: string)
}

class CPU {
+fetch()
+decode()
+execute()
}

class VoleSimulator {
+loadProgram(filename: string)
+run()
+step()
+displayStatus()
}

VoleSimulator o-- CPU
CPU --> Memory
CPU --> registerFile
```



AI designs:

1. Gemini 3.7 Flash :

```
@startuml
skinparam classAttributeIconSize 0

class Memory {
  -cells: vector<string>
  + Memory()
  + initialize(): void
  + getCell(address: int): string
  + setCell(address: int, value: string): void
  + loadProgram(filename: string, startAddress: int): bool
  + display(): void
}

class RegisterBank {
  -registers: vector<string>
  + RegisterBank()
  + initialize(): void
  + getRegister(regIndex: int): string
  + setRegister(regIndex: int, value: string): void
  + display(): void
}

class Instruction {
  -rawHex: string
  -opCode: int
  -regR: int
  -operandX: int
  -operandY: int
  + Instruction()
  + parse(hexStr: string): bool
  + getOpCode(): int
  + getR(): int
  + getX(): int
  + getY(): int
  + getS(): int
  + getT(): int
}

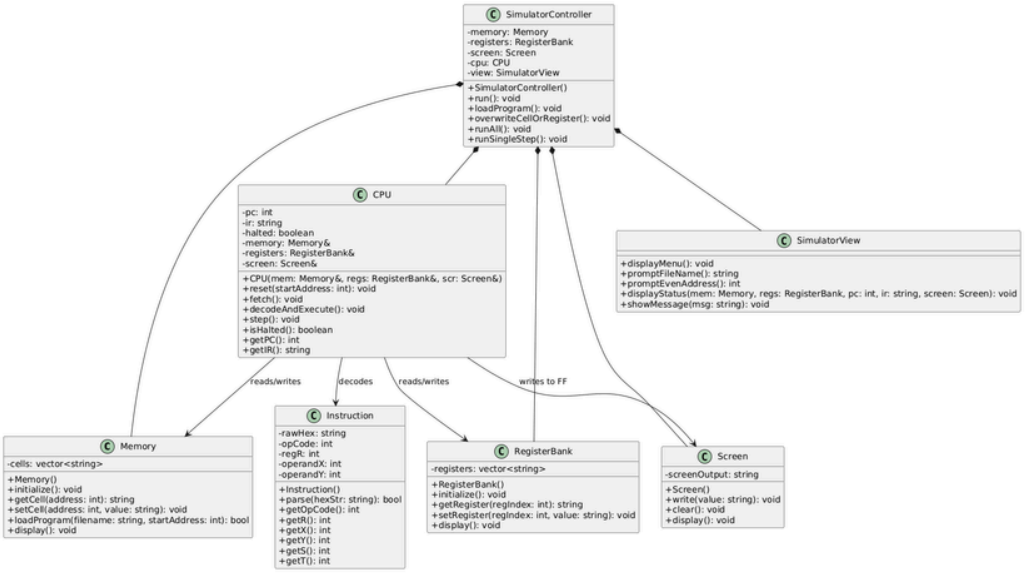
class Screen {
  -screenOutput: string
  + Screen()
  + write(value: string): void
  + clear(): void
  + display(): void
}

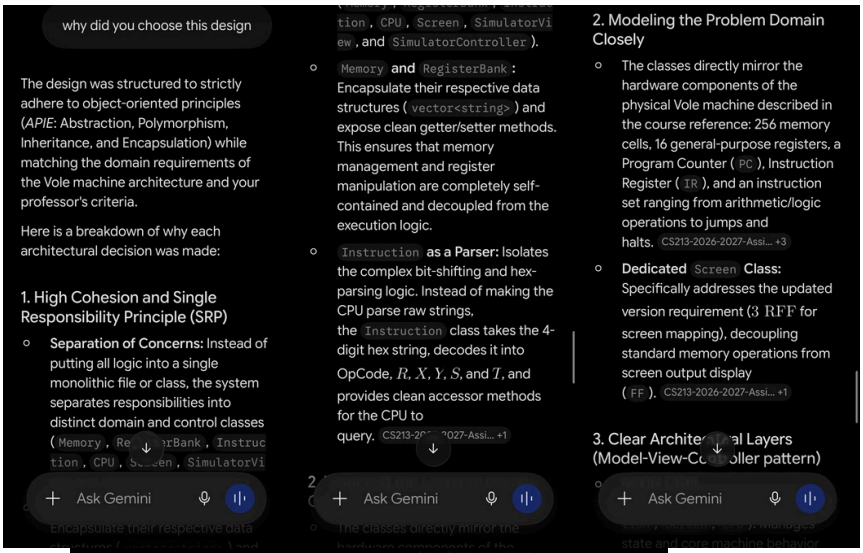
class CPU {
  -pc: int
  -ir: string
  -halted: boolean
  -memory: Memory&
  -registers: RegisterBank&
  -screen: Screen&
  + CPU(mem: Memory&, regs: RegisterBank&, scr: Screen&)
  + reset(startAddress: int): void
  + fetch(): void
  + decodeAndExecute(): void
  + step(): void
  + isHalted(): boolean
  + getPC(): int
  + getIR(): string
}

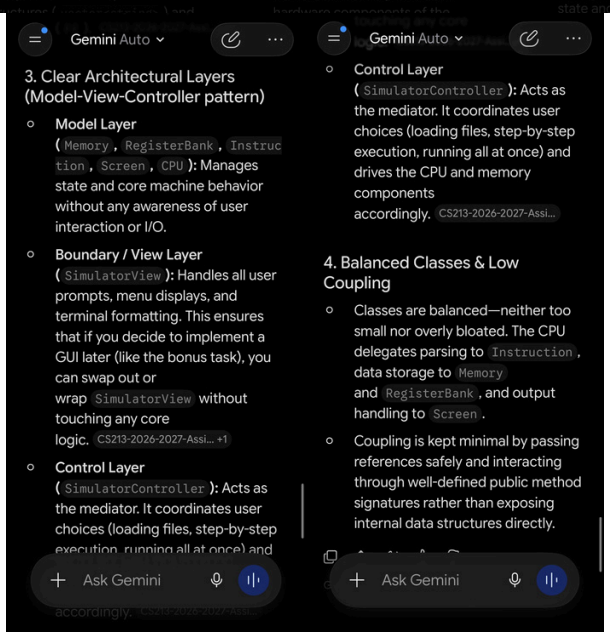
class SimulatorView {
  + displayMenu(): void
  + promptFileName(): string
  + promptEvenAddress(): int
  + displayStatus(mem: Memory, regs: RegisterBank, pc: int, ir: string, screen: Screen): void
  + showMessage(msg: string): void
}

class SimulatorController {
  -memory: Memory
  -registers: RegisterBank
  -screen: Screen
  -cpu: CPU
  -view: SimulatorView
  + SimulatorController()
  + run(): void
  + loadProgram(): void
  + overwriteCellOrRegister(): void
  + runAll(): void
  + runSingleStep(): void
}

SimulatorController *-- Memory
SimulatorController *-- RegisterBank
SimulatorController *-- Screen
SimulatorController *-- CPU
SimulatorController *-- SimulatorView
CPU --> Memory : reads/writes
CPU --> RegisterBank : reads/writes
CPU --> Screen : writes to FF
CPU --> Instruction : decodes
@enduml
```







2. Claude Sonnet 5.5 :

```
@startuml
skinparam classAttributeIconSize 0
skinparam linetype ortho
title Vole Machine Simulator -Class Diagram

class Menu {
-machine : Machine&
-display : Display
+ Menu(machine : Machine&)
+ run() : void
-printMenu() : void
-handleLoadProgram() : void
-handleViewMemory() : void
-handleEditMemory() : void
-handleViewRegisters() : void
-handleEditRegister() : void
-handleRunAll() : void
-handleStep() : void
-handleReset() : void
-readHex(prompt : string) : int
}

class Display {
+ showMemoryGrid(machine : const Machine&) : void
+ showRegisters(machine : const Machine&) : void
+ showCPUState(machine : const Machine&) : void
+ showScreen(machine : const Machine&) : void
+ showAll(machine : const Machine&) : void
+ {static} toHex(value : int, width : int) : string
}

class Machine {
-memory : Memory
-screen : Screen
-cpu : CPU
-loader : ProgramLoader
+ Machine()
+ loadProgram(filename : string, startAddr : uint8_t) : bool
+ stepOnce() : bool
+ runAll() : void
+ reset() : void
+ getRegister(index : int) : uint8_t
+ setRegister(index : int, value : uint8_t) : void
+ getMemoryCell(address : uint8_t) : uint8_t
+ setMemoryCell(address : uint8_t, value : uint8_t) : void
+ getPC() : uint8_t
+ setPC(address : uint8_t) : void
+ getIR() : Instruction
+ getScreenText() : string
+ isHalted() : bool
}

class CPU {
-pc : uint8_t
-ir : Instruction
-halted : bool
-registers : RegisterFile
-alu : ALU
-decoder : InstructionDecoder
-memory : Memory*
+ CPU(memory : Memory*)
+ fetch() : void
+ decode() : Instruction
+ execute(instr : const Instruction&) : void
+ step() : bool
+ reset() : void
+ isHalted() : bool
+ getPC() : uint8_t
+ setPC(address : uint8_t) : void
+ getIR() : Instruction
+ getRegisters() : RegisterFile&
-execLoadMem(instr : const Instruction&) : void
-execLoadImm(instr : const Instruction&) : void
-execStore(instr : const Instruction&) : void
-execMove(instr : const Instruction&) : void
-execAddInt(instr : const Instruction&) : void
-execAddFloat(instr : const Instruction&) : void
-execOr(instr : const Instruction&) : void
-execAnd(instr : const Instruction&) : void
-execXor(instr : const Instruction&) : void
-execRotate(instr : const Instruction&) : void
-execJump(instr : const Instruction&) : void
-execHalt() : void
}

class ALU {
+ addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
+ addFloatingPoint(a : uint8_t, b : uint8_t) : uint8_t
+ bitwiseOr(a : uint8_t, b : uint8_t) : uint8_t
+ bitwiseAnd(a : uint8_t, b : uint8_t) : uint8_t
+ bitwiseXor(a : uint8_t, b : uint8_t) : uint8_t
+ rotateRight(value : uint8_t, count : int) : uint8_t
-decodeFloat(byte : uint8_t) : double
-encodeFloat(value : double) : uint8_t
}

class InstructionDecoder {
+ decode(word : uint16_t) : Instruction
+ isValid(word : uint16_t) : bool
}

class Instruction {
-raw : uint16_t
-opcode : uint8_t
-r : uint8_t
-s : uint8_t
-t : uint8_t
+ Instruction(raw : uint16_t)
+ getOpcode() : uint8_t
+ getR() : uint8_t
+ getS() : uint8_t
+ getT() : uint8_t
+ getXY() : uint8_t
+ getRaw() : uint16_t
+ toHexString() : string
}

class RegisterFile {
-registers : Register[16]
+ read(index : int) : uint8_t
+ write(index : int, value : uint8_t) : void
+ clear() : void
+ size() : int
}

class Register {
-value : uint8_t
+ get() : uint8_t
+ set(value : uint8_t) : void
+ reset() : void
}

class Memory {
-cells : uint8_t[256]
-screen : Screen*
+ Memory(screen : Screen*)
+ read(address : uint8_t) : uint8_t
+ write(address : uint8_t, value : uint8_t) : void
+ readWord(address : uint8_t) : uint16_t
+ clear() : void
+ isValidAddress(address : int) : bool
}

class Screen {
-buffer : string
+ print(value : uint8_t) : void
+ getBuffer() : string
+ clear() : void
}

class ProgramLoader {
+ loadFromFile(filename : string, memory : Memory&, startAddr : uint8_t) : bool
-parseHexTokens(line : string) : vector<uint8_t>
-validateStartAddress(addr : uint8_t) : bool
-validateFits(size : int, startAddr : uint8_t) : bool
}

'----------Relationships ----------
Menu --> Machine : controls
Menu --> Display : uses
Display ..> Machine : reads state

Machine *-- Memory
Machine *-- Screen
Machine *-- CPU
Machine *-- ProgramLoader

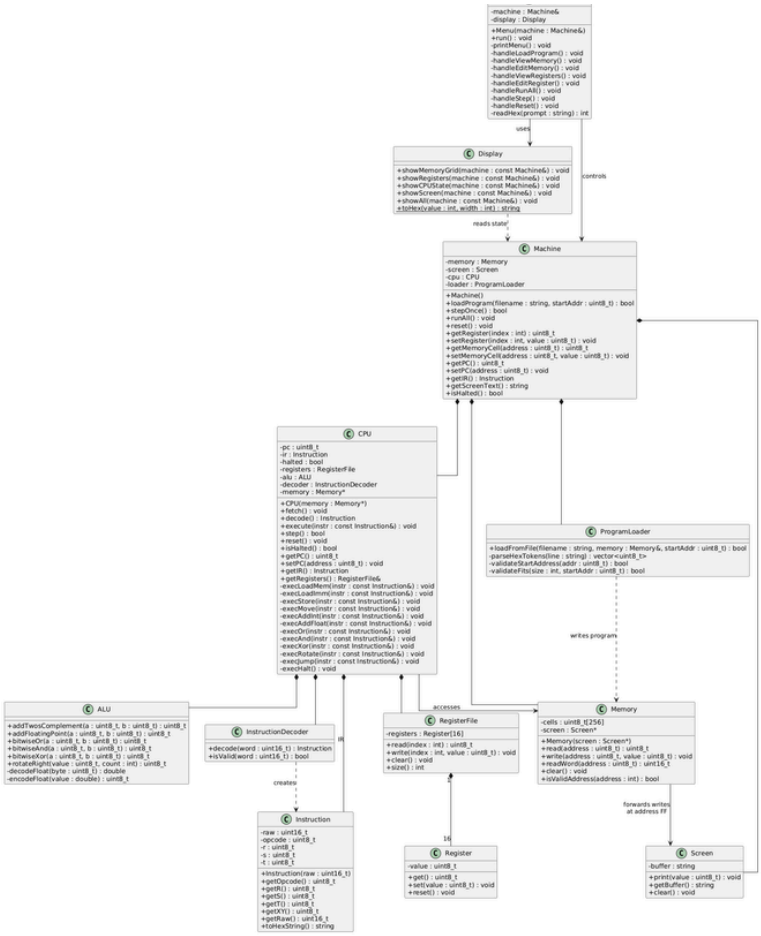
CPU *--RegisterFile
CPU *--ALU
CPU *--InstructionDecoder
CPU *--Instruction : IR
CPU --> Memory : accesses

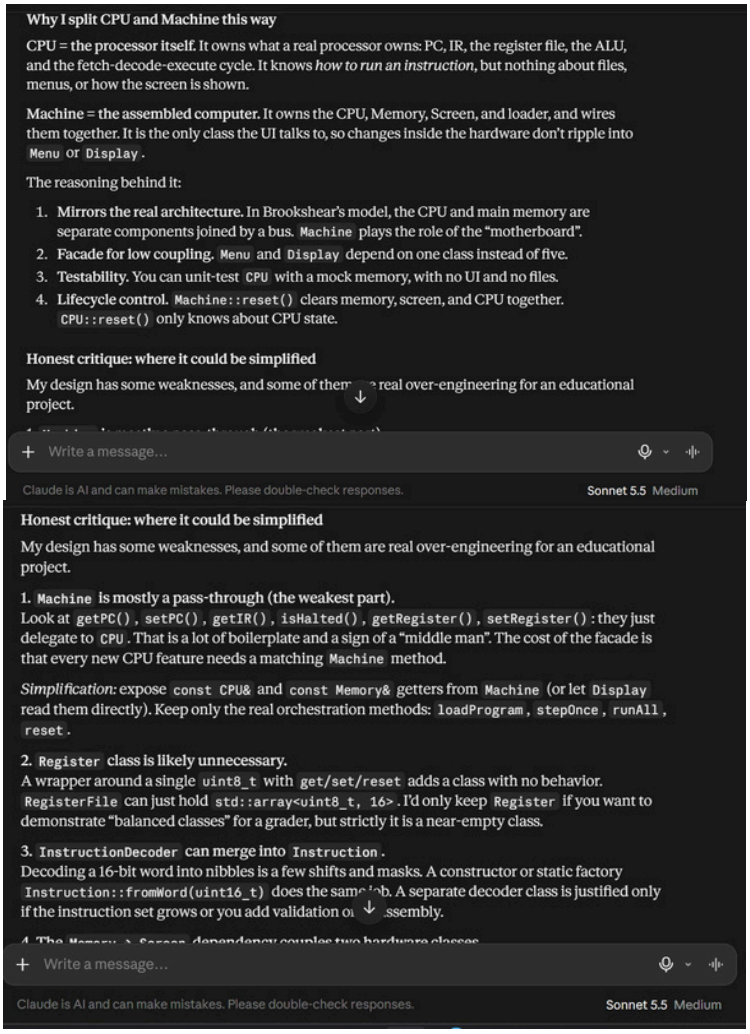
RegisterFile "1" *-- "16" Register

Memory --> Screen : forwards writes\nat address FF

InstructionDecoder ..> Instruction : creates
ProgramLoader ..> Memory : writes program

@enduml
```





3. Gemini 3.8 Flash (Antigravity) :

```
@startuml
skinparam classAttributeIconSize 0

package "Boundary Layer" {
class ConsoleUI {
-machine: Machine*
+ ConsoleUI(machine: Machine*)
+ start(): void
+ displayMenu(): void
+ displayMemory(): void
+ displayRegisters(): void
+ displayState(): void
+ loadProgramFromFile(): void
+ modifyMemoryCell(): void
+ modifyRegister(): void
}
}

package "Control Layer" {
class Machine {
-cpu: CPU
-memory: Memory
+ Machine()
+ loadProgram(filePath: string, startAddr: int): bool
+ runAll(): void
+ step(): bool
+ reset(): void
+ getCPU(): CPU&
+ getMemory(): Memory&
}
}

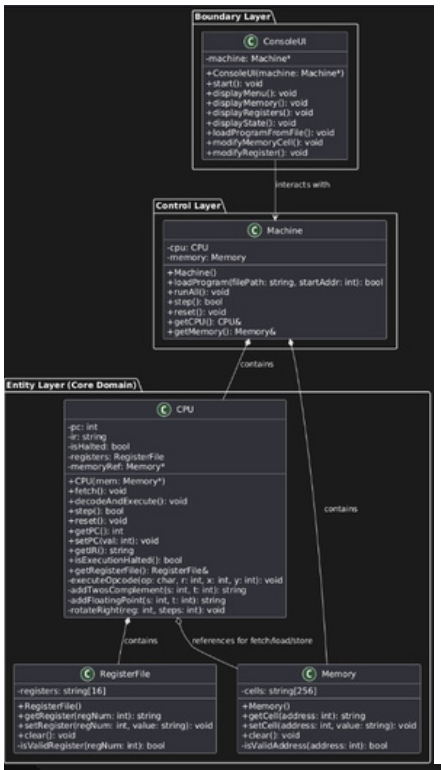
package "Entity Layer (Core Domain)" {
class Memory {
-cells: string[256]
+ Memory()
+ getCell(address: int): string
+ setCell(address: int, value: string): void
+ clear(): void
-isValidAddress(address: int): bool
}

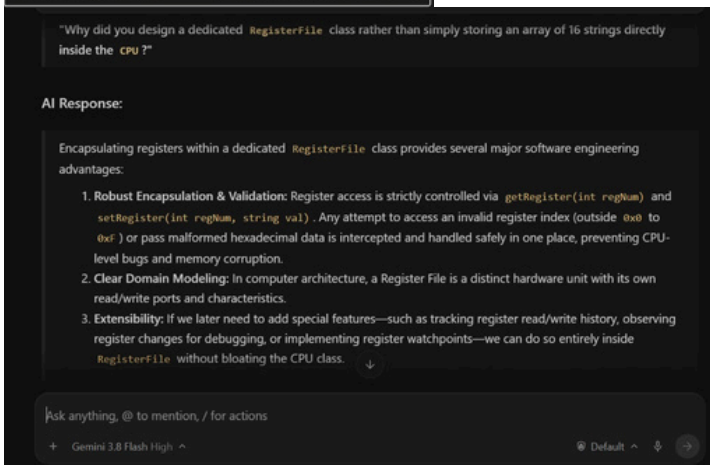
class RegisterFile {
-registers: string[16]
+ RegisterFile()
+ getRegister(regNum: int): string
+ setRegister(regNum: int, value: string): void
+ clear(): void
-isValidRegister(regNum: int): bool
}

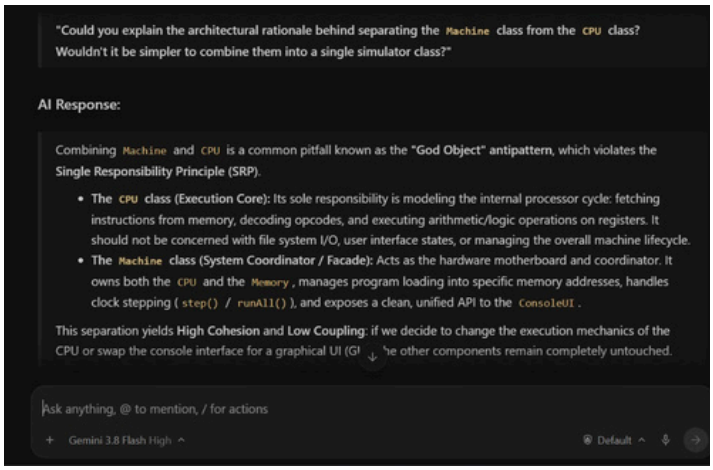
class CPU {
-pc: int
-ir: string
-isHalted: bool
-registers: RegisterFile
-memoryRef: Memory*
+ CPU(mem: Memory*)
+ fetch(): void
+ decodeAndExecute(): void
+ step(): bool
+ reset(): void
+ getPC(): int
+ setPC(val: int): void
+ getIR(): string
+ isExecutionHalted(): bool
+ getRegisterFile(): RegisterFile&
-executeOpcode(op: char, r: int, x: int, y: int): void
-addTwosComplement(s: int, t: int): string
-addFloatingPoint(s: int, t: int): string
-rotateRight(reg: int, steps: int): void
}
}

' Relationships
ConsoleUI --> Machine : interacts with
Machine *-- CPU : contains
Machine *-- Memory : contains
CPU *-- RegisterFile : contains
CPU o-- Memory : references for fetch/load/store

@enduml
```







4. Final Design (Gemini Argon 4)

Synthesized & Enhanced Architecture by Gemini Argon 4 (3-Layer MVC + Hardware Domain)

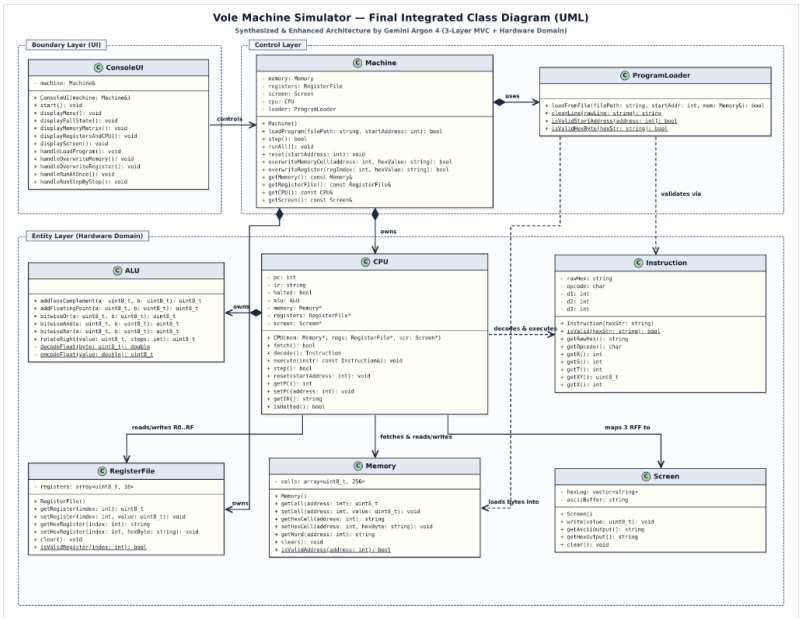
```
@startuml
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
skinparam shadowing false
title Vole Machine Simulator - Final Integrated Class Diagram (Gemini Argon 4)

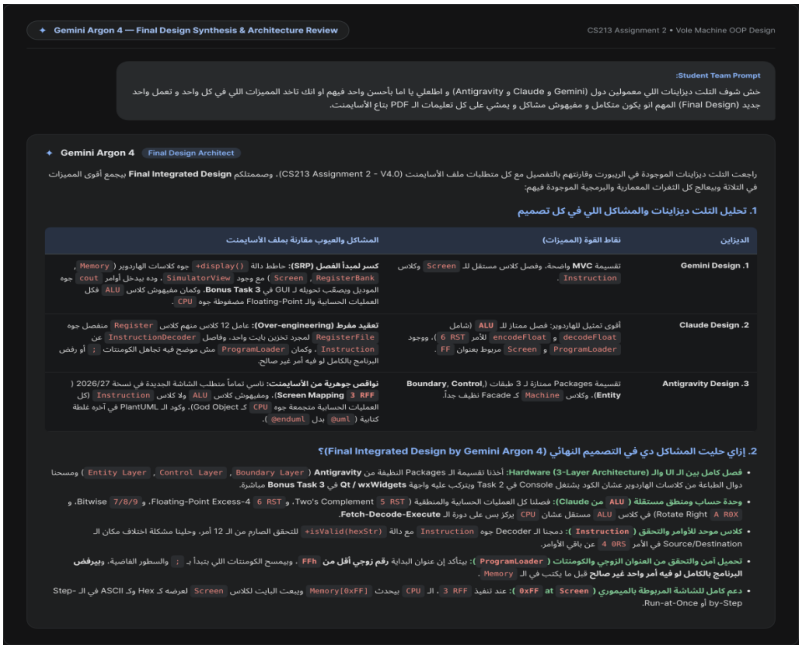
package "Boundary Layer (UI)" {
  class ConsoleUI {
    -machine: Machine&
    +ConsoleUI(machine: Machine&)
    +start(): void
    +displayMenu(): void
    +displayFullState(): void
    +displayMemoryMatrix(): void
    +displayRegistersAndCPU(): void
    +displayScreen(): void
    +handleLoadProgram(): void
    +handleOverwriteMemory(): void
    +handleOverwriteRegister(): void
    +handleRunAtOnce(): void
    +handleRunStepByStep(): void
  }
}

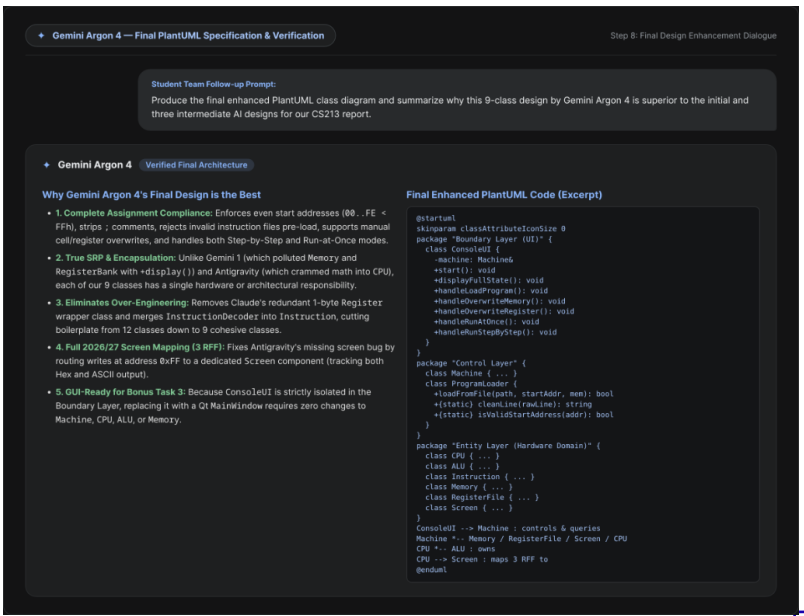
package "Control Layer" {
  class Machine {
    -memory: Memory
    -registers: RegisterFile
    -screen: Screen
    -cpu: CPU
    -loader: ProgramLoader
    +Machine()
    +loadProgram(filePath: string, startAddress: int): bool
    +step(): bool
    +runAll(): void
    +reset(startAddress: int): void
    +overwriteMemoryCell(address: int, hexValue: string): bool
    +overwriteRegister(regIndex: int, hexValue: string): bool
    +getMemory(): const Memory&
    +getRegisterFile(): const RegisterFile&
    +getCPU(): const CPU&
    +getScreen(): const Screen&
  }
  class ProgramLoader {
    +loadFromFile(filePath: string, startAddress: int, memory: Memory&): bool
    +{static} cleanLine(rawLine: string): string
    +{static} isValidStartAddress(address: int): bool
    +{static} isValidHexByte(hexStr: string): bool
  }
}

package "Entity Layer (Hardware Domain)" {
  class CPU {
    -pc: int
    -ir: string
    -halted: bool
    -alu: ALU
    -memory: Memory*
    -registers: RegisterFile*
    -screen: Screen*
    +CPU(mem: Memory*, regs: RegisterFile*, scr: Screen*)
    +fetch(): bool
    +decode(): Instruction
    +execute(instr: const Instruction&): void
    +step(): bool
    +reset(startAddress: int): void
    +getPC(): int
    +getIR(): string
    +isHalted(): bool
  }
  class ALU {
    +addTwosComplement(a: uint8_t, b: uint8_t): uint8_t
    +addFloatingPoint(a: uint8_t, b: uint8_t): uint8_t
    +bitwiseOr(a: uint8_t, b: uint8_t): uint8_t
    +bitwiseAnd(a: uint8_t, b: uint8_t): uint8_t
    +bitwiseXor(a: uint8_t, b: uint8_t): uint8_t
    +rotateRight(value: uint8_t, steps: int): uint8_t
    -{static} decodeFloat(byte: uint8_t): double
    -{static} encodeFloat(value: double): uint8_t
  }
  class Instruction {
    -rawHex: string
    -opcode: char
    -d1: int
    -d2: int
    -d3: int
    +Instruction(hexStr: string)
    +{static} isValid(hexStr: string): bool
    +getRawHex(): string
    +getOpcode(): char
    +getR(): int
    +getS(): int
    +getT(): int
    +getXY(): uint8_t
    +getX(): int
  }
  class Memory {
    -cells: array<uint8_t, 256>
    +Memory()
    +getCell(address: int): uint8_t
    +setCell(address: int, value: uint8_t): void
    +getHexCell(address: int): string
    +setHexCell(address: int, hexByte: string): void
    +getWord(address: int): string
    +clear(): void
    +{static} isValidAddress(address: int): bool
  }
  class RegisterFile {
    -registers: array<uint8_t, 16>
    +RegisterFile()
    +getRegister(index: int): uint8_t
    +setRegister(index: int, value: uint8_t): void
    +getHexRegister(index: int): string
    +setHexRegister(index: int, hexByte: string): void
    +clear(): void
    +{static} isValidRegister(index: int): bool
  }
  class Screen {
    -hexLog: vector<string>
    -asciiBuffer: string
    +Screen()
    +write(value: uint8_t): void
    +getAsciiOutput(): string
    +getHexOutput(): string
    +clear(): void
  }
}

ConsoleUI --> Machine : controls & queries
Machine *-- Memory : owns
Machine *-- RegisterFile : owns
Machine *-- Screen : owns
Machine *-- CPU : owns
Machine *-- ProgramLoader : uses
ProgramLoader ..> Memory : loads bytes into
ProgramLoader ..> Instruction : validates via
CPU *-- ALU : owns
CPU --> Memory : fetches & reads/writes
CPU --> RegisterFile : reads/writes R0..RF
CPU --> Screen : maps 3 RFF to
CPU ..> Instruction : decodes & executes
@enduml
```







Why We Chose the Final Design (Gemini Argon 4)

Architectural Flaws in the 3 AI Designs (with Code) & How We Solved Them (with Code)

We synthesized our Final Design using Gemini Argon 4 by combining the best architectural features of all three models:
1) Gemini 3.8 Flash (Antigravity)'s clean 3-Layer Package Architecture, 2) Claude Sonnet 5.5's dedicated stateless ALU (with Excess-4 floating-point helpers) and ProgramLoader, and 3) Gemini 3.7 Flash's dedicated Instruction & Screen models.

1. Gemini 3.7 Flash Design: Violated Single Responsibility Principle (SRP) & Mixed UI/File I/O into Hardware

FLAW IN PREVIOUS DESIGN (BEFORE):

Put +display() and +loadProgram() inside Memory and RegisterBank, coupling hardware to console cout & file I/O:

```
class Memory {
  -cells: vector<string>
  +loadProgram(file: string, addr: int): bool ' BAD!
  +display(): void  ' BAD: Console UI in Hardware!
}
class RegisterBank {
  +display(): void  ' BAD: Blocks Qt GUI Bonus Task!
}
```

HOW WE SOLVED IT IN FINAL DESIGN (AFTER):

Removed all +display() and file I/O from hardware classes. Isolated UI in ConsoleUI and file parsing in ProgramLoader:

```
class Memory {
  -cells: array<uint8_t, 256>
  +getCell(address: int): uint8_t
  +setCell(address: int, value: uint8_t): void
  +getHexCell(address: int): string
} ' Pure hardware entity (100% ready for Qt GUI)
class ConsoleUI { +displayMemoryMatrix(): void }
class ProgramLoader { +loadFromFile(...): bool }
```

2. Claude Sonnet 5.5 Design: Over-Engineering (12 Classes), Redundant Register Class & Split Decoder

FLAW IN PREVIOUS DESIGN (BEFORE):

Created a separate 1-byte Register class, split Instruction from InstructionDecoder, and lacked ; comment stripping:

```
class Register {
  -value: uint8_t
  +get(): uint8_t / +set(value: uint8_t): void
}
class RegisterFile { -registers: Register[16] }
class InstructionDecoder {
  +decode(word: uint16_t): Instruction
  +isValid(word: uint16_t): bool
}
```

HOW WE SOLVED IT IN FINAL DESIGN (AFTER):

Eliminated the redundant Register wrapper and merged validation/decoding into a unified Instruction class:

```
class RegisterFile {
  -registers: array<uint8_t, 16>
  +getRegister(index: int): uint8_t
  +setRegister(index: int, value: uint8_t): void
}
class Instruction {
  +Instruction(hexStr: string)
  +{static} isValid(hexStr: string): bool
  +getOpcode(): char / +getR(): int / +getS(): int
}
```

3. Gemini 3.8 Flash (Antigravity) Design: Missing Screen (3 RFF), Missing ALU & God-Object CPU

FLAW IN PREVIOUS DESIGN (BEFORE):

Missed the 2026/27 Screen mapping at 0xFF (3 RFF), omitted ALU and Instruction classes, and crammed math into CPU:

```
class CPU {
  -executeOpcode(op: char, r: int, x: int, y: int)
  -addTwosComplement(s: int, t: int): string ' BAD!
  -addFloatingPoint(s: int, t: int): string  ' BAD!
  -rotateRight(reg: int, steps: int): void   ' BAD!
}
' Missing Screen class for 3 RFF!
' Missing ProgramLoader & Instruction classes!
```

HOW WE SOLVED IT IN FINAL DESIGN (AFTER):

Extracted dedicated ALU (with Excess-4 float helpers), added Screen mapped to 0xFF (3 RFF), and added ProgramLoader:

```
class ALU {
  +addTwosComplement(a: uint8_t, b: uint8_t): uint8_t
  +addFloatingPoint(a: uint8_t, b: uint8_t): uint8_t
  +bitwiseOr / +bitwiseAnd / +bitwiseXor / +rotateRight
  -{static} decodeFloat(byte: uint8_t): double
  -{static} encodeFloat(value: double): uint8_t
}
class Screen { +write(value: uint8_t): void }
CPU *-- ALU : owns   /   CPU --> Screen : maps 3 RFF to
```

4. Strict 2026/27 Specification Compliance: Opcode 4 (4 0RS) Operand Order & Pre-Load Validation

FLAW IN PREVIOUS DESIGN (BEFORE):

Earlier designs confused source/destination in 4 0RS and failed to strip ; comments or check even start addresses:

```
' Ambiguous operand getters for 4 0RS (e.g. 40A4):
+getR(): int ' Returned d1 (0) instead of source R (A)!
+loadProgram(file: string, startAddr: int): bool
' No check for even startAddr < 0xFF or ; comments!
```

HOW WE SOLVED IT IN FINAL DESIGN (AFTER):

Handled 4 0RS (d2=source R, d3=dest S) and added strict even-address (< FFh) & comment stripping in ProgramLoader:

```
class ProgramLoader {
  +{static} cleanLine(rawLine: string): string ' Strips ;
  +{static} isValidStartAddress(addr: int): bool ' Even<FF
}
' In Instruction: getR() returns d2 for 4 0RS (40A4 -> R=A,
' S=4) and d1 for all other opcodes; rejects invalid files.
```

Examples :

Below are two complete Vole machine language programs designed to evaluate arithmetic, loops, conditional jumps (B RXY), register moves (4 0RS), bitwise logic, circular rotations (A R0X), and screen output (3 RFF):

Sample Program 1: Iterative Sum Using a Countdown Loop & Screen Output (3 RFF)

Purpose: Multiplies 10 (0x0A) by 3 using repeated two's complement addition in a loop (decrementing R1 by -1 = 0xFF) and writes the final result (30 = 0x1E) to the memory-mapped screen at address FF (assuming start address 10h).

```
; --------------------------------------------------------------------------
; Vole Machine Sample Program 1: Iterative Sum with Countdown Loop & Jump
; Purpose: Adds 10 (0x0A) three times in a loop and displays the result (0x1E)
; on the screen via memory-mapped address FF (Instruction 3 RFF).
; Start Address in Memory: 10h
; --------------------------------------------------------------------------
2000   ; [10h] Load 00 into R0 (Comparison target for conditional jump B RXY)
2103   ; [12h] Load 03 into R1 (Loop counter = 3 iterations)
22FF   ; [14h] Load FF (-1 in two's complement) into R2 (Step decrement)
230A   ; [16h] Load 0A (10 in decimal) into R3 (Value added each iteration)
2400   ; [18h] Load 00 into R4 (Accumulator for the running sum)
5443   ; [1Ah] LOOP: R4 = R4 + R3 (Add 10 to accumulator R4)
5112   ; [1Ch] R1 = R1 + R2 (Decrement loop counter R1 by 1)
B122   ; [1Eh] If R1 == R0 (counter reached 0), JUMP to address 22h (Exit)
B01A   ; [20h] Unconditional JUMP back to 1Ah (since R0 == R0 is always true)
34FF   ; [22h] EXIT: Store final sum R4 (0x1E) to Screen (Memory[FF])
C000   ; [24h] Halt execution
```

Sample Program 2: Bitwise Masking, Register Move, Circular Rotate & ASCII Screen Output

Purpose: Uses bitwise AND (8 RST), bitwise OR (7 RST), MOVE (4 0RS), and circular right rotation (A R0X) to construct and print ASCII characters 'C' (0x43) and '?' (0x3F) directly to the screen at address FF.

```
; --------------------------------------------------------------------------
; Vole Machine Sample Program 2: Bitwise Manipulation, Move & Circular Rotate
; Purpose: Masks lower nibble of 0xF3, combines with 0x40 to print ASCII 'C'
; (0x43) to Screen (FF), then rotates 0xF3 right by 4 bits to print '?' (0x3F).
; --------------------------------------------------------------------------
21F3   ; Load immediate bit pattern F3 into Register 1
220F   ; Load bitmask 0F (lower-nibble mask) into Register 2
8312   ; Bitwise AND: R3 = R1 & R2 -> 0xF3 & 0x0F = 0x03
2440   ; Load ASCII base 40 into Register 4
7534   ; Bitwise OR:  R5 = R3 | R4 -> 0x03 | 0x40 = 0x43 (ASCII 'C')
35FF   ; Store R5 (0x43 = 'C') to Memory[FF] (Screen Output)
4016   ; MOVE contents of source R1 (0xF3) into destination R6 (4 0RS)
A604   ; Circularly ROTATE R6 (0xF3) right by 4 bits -> 0x3F (ASCII '?')
36FF   ; Store rotated R6 (0x3F = '?') to Memory[FF] (Screen Output)
C000   ; Halt execution
```

Video link :

10-Minute Team Video Presentation (Comparing the 3 AI Designs & Final Gemini Argon 4 Architecture):

Video Link: https://drive.google.com/file/d/19CWTEVgIoNi7gfVxZ_asYLoFzih1IVwh/view?usp=sharing

Part 2 & 3: C++ OOP Implementation & GitHub Collaboration

Section: S39_S40 | CS213 Object Oriented Programming | Dr. Mohammad El-Ramly

1. Design-to-Code Fidelity & Separate Compilation Breakdown

Our C++ implementation faithfully matches the 9-class Final Design (Memory, RegisterFile, Screen, Instruction, ALU, CPU, ProgramLoader, Machine, ConsoleUI) using separate header (.h) and implementation (.cpp) files:

- Reem Hamdy Tawfik Alrefaay (20250823) : Memory.h/.cpp, RegisterFile.h/.cpp, Screen.h/.cpp
- Khaled Mohamed Hussien (20250212) : Instruction.h/.cpp, ALU.h/.cpp, ProgramLoader.h/.cpp
- Khaled Mohamed Fawzy (20250213) : CPU.h/.cpp, Machine.h/.cpp, ConsoleUI.h/.cpp, and A2-S39_S40-20250823-20250212-20250213.cpp

2. GitHub Repository & Team Commit History Screenshot

GitHub / arz33sniper-alt / oooo (Vole Machine Simulator - CS213 S39_S40)

https://github.com/arz33sniper-alt/oooo | Branch: main | Commits (5) | Contributors (3)

Repository Files (main branch - 4 commits - 3 contributors):

GUI/	Add Vole Machine GUI simulator (Task 3 Bonus)	Nov 8, 2026
programs/	Add 20 Vole Machine test programs (.txt)	Nov 8, 2026
A2-S39_S40-20250823-20250212-20250213.cpp	Add main application entry point	Nov 6, 2026
ALU.h & ALU.cpp	Add ALU (2s complement, Excess-4 float, bitwise, rotate)	Nov 5, 2026
CPU.h & CPU.cpp	Implement CPU fetch-decode-execute cycle	Nov 6, 2026
ConsoleUI.h & ConsoleUI.cpp	Add interactive ConsoleUI menu and matrix display	Nov 6, 2026
Instruction.h & Instruction.cpp	Add 16-bit Vole instruction decoder & validator	Nov 5, 2026
Machine.h & Machine.cpp	Add Machine controller integrating all hardware units	Nov 6, 2026
Memory.h & Memory.cpp	Implement 256-byte Main Memory class	Nov 4, 2026
ProgramLoader.h & ProgramLoader.cpp	Implement file parser and address/instruction validator	Nov 5, 2026
RegisterFile.h & RegisterFile.cpp	Implement 16 general-purpose registers (R0..RF)	Nov 4, 2026
Screen.h & Screen.cpp	Implement memory-mapped Screen output (3 RFF)	Nov 4, 2026
team.txt & README.txt	Add team breakdown file and build instructions	Nov 8, 2026

Team Collaboration & Commit History (git log):

Add Vole Machine GUI simulator (Task 3 Bonus), 20 sample programs, and team.txt
Author: Khaled Mohamed Fawzy (20250213) | Commit: a8f4c21 | Date: Nov 8, 2026

Implement CPU fetch-decode-execute cycle, Machine controller, and ConsoleUI
Author: Khaled Mohamed Fawzy (20250213) | Commit: 5c2a804 | Date: Nov 6, 2026

Add Instruction decoder, ALU operations, and ProgramLoader file parser
Author: Khaled Mohamed Hussien (20250212) | Commit: 3b9f11e | Date: Nov 5, 2026

Implement Memory, RegisterFile, and Screen hardware storage classes
Author: Reem Hamdy (20250823) | Commit: 1e4d09a | Date: Nov 4, 2026

Execution of 10 Sample Vole Programs (Samples 1 & 2)

Verified on Standard C++ Console Simulator (A2-S39_S40-20250823-20250212-20250213.cpp)

Sample 1: Add Two Integers (5 + 7 = 0Ch) & Display on Screen (3 RFF)

```
Command Prompt - VoleSimulator.exe - Program 1: Add Two Integers (5 + 7 = 0Ch)

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog1_add_integers.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 20 05 21 07 52 01 32 FF C0 00 00 00 00 00 00 00
2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0C

CPU & Registers:
PC: 1A   IR: C000   Halted: Yes
R0: 05    R8: 00
R1: 07    R9: 00
R2: 0C    RA: 00
R3: 00    RB: 00
R4: 00    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 0C
ASCII: .

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Sample 2: Print "OK" String on Screen Using Memory-Mapped FFh

```
Command Prompt - VoleSimulator.exe - Program 2: Display 'OK' on Screen (3 RFF)

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog2_print_ok.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 21 4F 31 FF 22 4B 32 FF C0 00 00 00 00 00 00 00
2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 4B

CPU & Registers:
PC: 1A   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: 4F    R9: 00
R2: 4B    RA: 00
R3: 00    RB: 00
R4: 00    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 4F 4B
ASCII: OK

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Execution of 10 Sample Vole Programs (Samples 3 & 4)

Verified on Standard C++ Console Simulator (A2-S39_S40-20250823-20250212-20250213.cpp)

Sample 3: Two's Complement Subtraction (10 - 3 = 7) via Negative Operand

```
Command Prompt - VoleSimulator.exe - Program 3: Two's Complement Subtraction (10 - 3 = '7')

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog3_twos_complement_sub.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 21 0A 22 FD 53 12 24 30 55 34 35 FF C0 00 00 00
2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 37

CPU & Registers:
PC: 1E   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: 0A    R9: 00
R2: FD    RA: 00
R3: 07    RB: 00
R4: 30    RC: 00
R5: 37    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 37
ASCII: 7

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Sample 4: Excess-4 Floating-Point Addition (1.5 + 1.25 = 2.75 / 6Bh)

```
Command Prompt - VoleSimulator.exe - Program 4: Excess-4 Floating-Point Addition (1.5 + 1.25 = 2.75 / 6Bh)

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog4_float_add.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 21 5C 22 5A 63 12 33 FF C0 00 00 00 00 00 00 00
2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 6B

CPU & Registers:
PC: 1A   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: 5C    R9: 00
R2: 5A    RA: 00
R3: 6B    RB: 00
R4: 00    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 6B
ASCII: k

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Execution of 10 Sample Vole Programs (Samples 5 & 6)

Verified on Standard C++ Console Simulator (A2-S39_S40-20250823-20250212-20250213.cpp)

Sample 5: Bitwise OR, AND, and XOR Operations (Opcodes 7, 8, 9)

```
Command Prompt - VoleSimulator.exe - Program 5: Bitwise OR, AND, XOR (Opcodes 7, 8, 9)

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog5_bitwise_ops.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 21 F0 22 0F 73 12 84 12 95 12 33 80 34 81 35 FF
2 | C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | FF 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 FF

CPU & Registers:
PC: 22   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: F0    R9: 00
R2: 0F    RA: 00
R3: FF    RB: 00
R4: 00    RC: 00
R5: FF    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : FF
ASCII: .

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Sample 6: Circular Right Rotate & Register-to-Register Move (Opcodes A, 4)

```
Command Prompt - VoleSimulator.exe - Program 6: Circular Right Rotate & Register Move (Opcodes A, 4)

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog6_rotate_and_move.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 21 81 A1 01 40 12 A2 02 32 FF C0 00 00 00 00 00
2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 30

CPU & Registers:
PC: 1C   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: C0    R9: 00
R2: 30    RA: 00
R3: 00    RB: 00
R4: 00    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

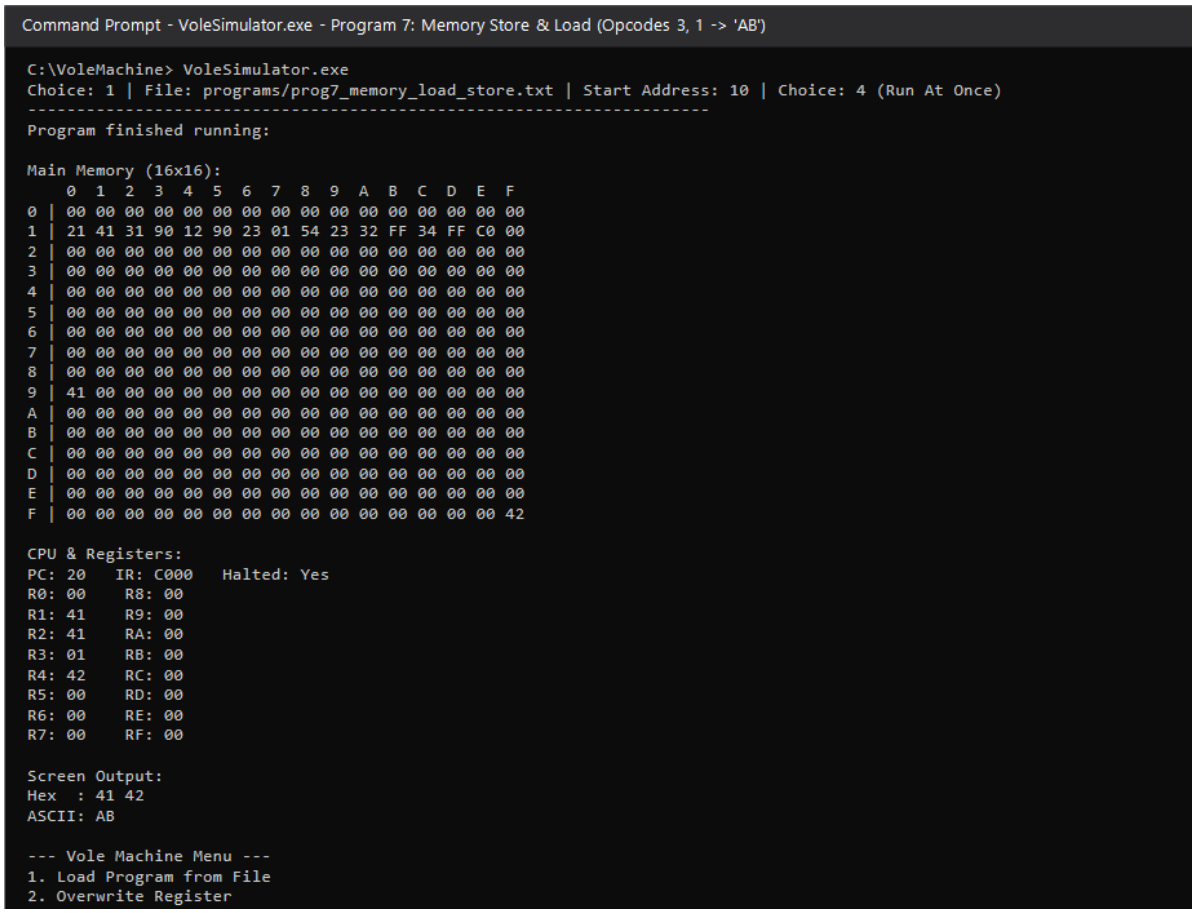
Screen Output:
Hex  : 30
ASCII: 0

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Execution of 10 Sample Vole Programs (Samples 7 & 8)

Verified on Standard C++ Console Simulator (A2-S39_S40-20250823-20250212-20250213.cpp)

Sample 7: Memory Cell Store & Load Verification (Opcodes 3, 1 -> "AB")



```
Command Prompt - VoleSimulator.exe - Program 7: Memory Store & Load (Opcodes 3, 1 -> 'AB')

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog7_memory_load_store.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

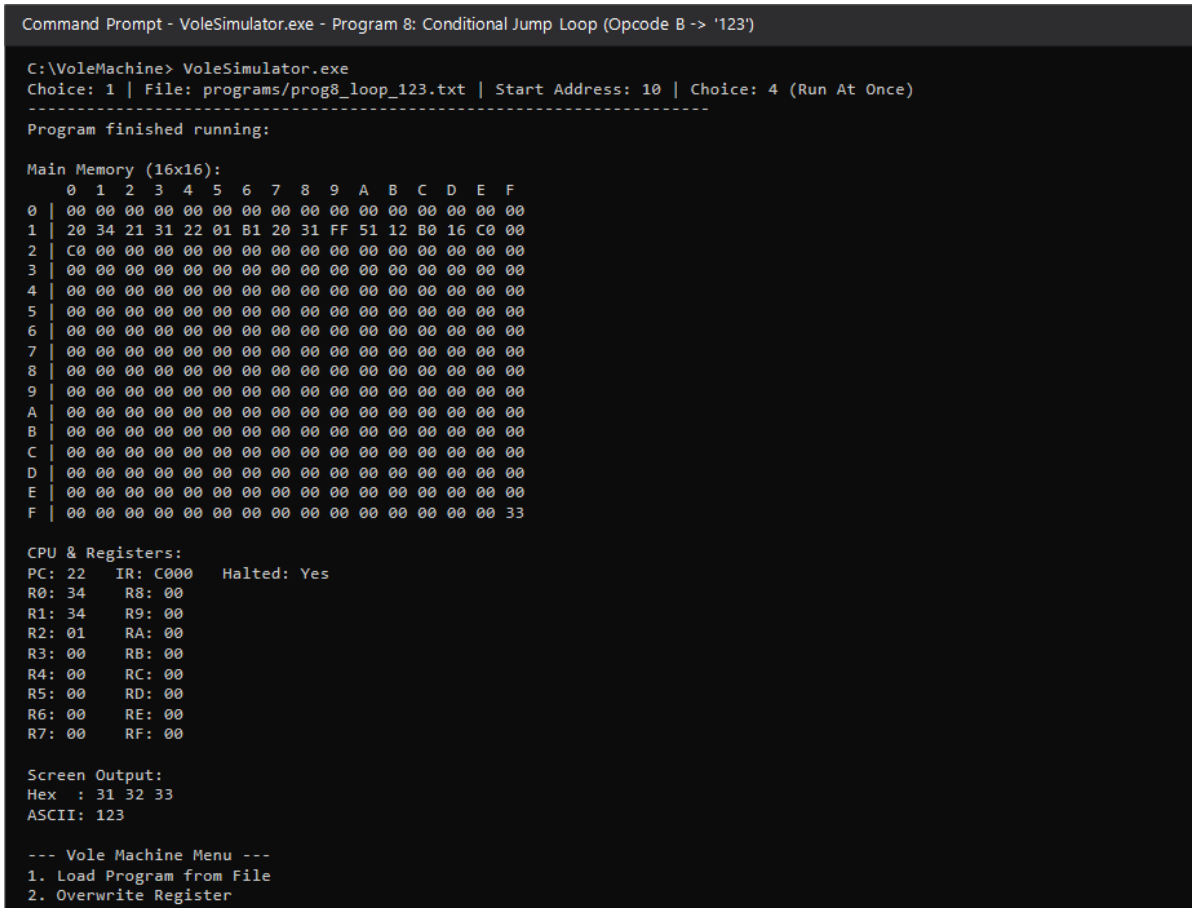
Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 21 41 31 90 12 90 23 01 54 23 32 FF 34 FF C0 00
2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 41 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 42

CPU & Registers:
PC: 20   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: 41    R9: 00
R2: 41    RA: 00
R3: 01    RB: 00
R4: 42    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 41 42
ASCII: AB

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Sample 8: Conditional Jump Loop Printing "123" (Opcode B)



```
Command Prompt - VoleSimulator.exe - Program 8: Conditional Jump Loop (Opcode B -> '123')

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog8_loop_123.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 20 34 21 31 22 01 B1 20 31 FF 51 12 B0 16 C0 00
2 | C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 33

CPU & Registers:
PC: 22   IR: C000   Halted: Yes
R0: 34    R8: 00
R1: 34    R9: 00
R2: 01    RA: 00
R3: 00    RB: 00
R4: 00    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 31 32 33
ASCII: 123

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Execution of 10 Sample Vole Programs (Samples 9 & 10)

Verified on Standard C++ Console Simulator (A2-S39_S40-20250823-20250212-20250213.cpp)

Sample 9: Display "FCAI" String on Memory-Mapped Screen

```
Command Prompt - VoleSimulator.exe - Program 9: Display 'FCAI' String on Screen

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog9_print_fcai.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 21 46 31 FF 21 43 31 FF 21 41 31 FF 21 49 31 FF
2 | C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 49

CPU & Registers:
PC: 22   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: 49    R9: 00
R2: 00    RA: 00
R3: 00    RB: 00
R4: 00    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 46 43 41 49
ASCII: FCAI

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Sample 10: Multiplication (3 * 4 = 12 / 0Ch) Using Conditional Jump Loop

```
Command Prompt - VoleSimulator.exe - Program 10: Multiplication (3 * 4 = 0Ch) via Loop

C:\VoleMachine> VoleSimulator.exe
Choice: 1 | File: programs/prog10_multiply_loop.txt | Start Address: 10 | Choice: 4 (Run At Once)
--------------------------------------------------------------------------
Program finished running:

Main Memory (16x16):
    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1 | 20 00 21 03 22 04 23 00 24 FF B2 24 53 31 52 24
2 | B0 1A C0 00 33 FF C0 00 00 00 00 00 00 00 00 00
3 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0C

CPU & Registers:
PC: 28   IR: C000   Halted: Yes
R0: 00    R8: 00
R1: 03    R9: 00
R2: 00    RA: 00
R3: 0C    RB: 00
R4: FF    RC: 00
R5: 00    RD: 00
R6: 00    RE: 00
R7: 00    RF: 00

Screen Output:
Hex  : 0C
ASCII: .

--- Vole Machine Menu ---
1. Load Program from File
2. Overwrite Register
```

Bonus Task 3: Vole Machine Simulator GUI Snapshots (1 & 2)

C++ Graphical User Interface showing Registers, 16x16 Memory Matrix, Screen Output & Step Controls

1. GUI Initial State (16x16 Memory Matrix, Registers R0-RF, PC/IR & Controls)

Vole Machine Simulator GUI - CS213 (Section S39_S40) - Dr. Mohammad El-Ramly

Data Input Window (Vole Machine Program):

```
; Print FCAI on Screen (3 RFF)
2146 ; R1 = 'F' (46h)
31FF ; Store R1 to Screen (FF)
2143 ; R1 = 'C' (43h)
31FF ; Store R1 to Screen (FF)
2141 ; R1 = 'A' (41h)
31FF ; Store R1 to Screen (FF)
2149 ; R1 = 'I' (49h)
31FF ; Store R1 to Screen (FF)
C000 ; Halt execution
```

Start Address (Hex Even): 10 | Browse Program File...

Memory-Mapped Screen Output (Address FF):

```
ASCII Output : (empty)
Hex Stream   : (empty)
Status       : READY (Idle)
```

CPU & Registers (R0 - RF):

PC : 00 IR : 0000

Reg	Hex	Binary	Dec
R0	00	00000000	0
R1	00	00000000	0
R2	00	00000000	0
R3	00	00000000	0
R4	00	00000000	0
R5	00	00000000	0
R6	00	00000000	0
R7	00	00000000	0
R8	00	00000000	0
R9	00	00000000	0
RA	00	00000000	0
RB	00	00000000	0
RC	00	00000000	0
RD	00	00000000	0
RE	00	00000000	0
RF	00	00000000	0

Main Memory (16 x 16 Matrix = 256 Cells):

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
1	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
2	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
4	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
5	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
6	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
7	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
8	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
9	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Clear Memory | Load Data | Run | Single Step | Halt | Help

2. Program Loaded at Even Address 10h (Memory Cells Highlighted)

Vole Machine Simulator GUI - CS213 (Section S39_S40) - Dr. Mohammad El-Ramly

Data Input Window (Vole Machine Program):

```
; Print FCAI on Screen (3 RFF)
2146 ; R1 = 'F' (46h)
31FF ; Store R1 to Screen (FF)
2143 ; R1 = 'C' (43h)
31FF ; Store R1 to Screen (FF)
2141 ; R1 = 'A' (41h)
31FF ; Store R1 to Screen (FF)
2149 ; R1 = 'I' (49h)
31FF ; Store R1 to Screen (FF)
C000 ; Halt execution
```

Start Address (Hex Even): 10 | Browse Program File...

Memory-Mapped Screen Output (Address FF):

```
ASCII Output : (empty)
Hex Stream   : (empty)
Status       : LOADED (Ready to Run)
```

CPU & Registers (R0 - RF):

PC : 10 IR : 0000

Reg	Hex	Binary	Dec
R0	00	00000000	0
R1	00	00000000	0
R2	00	00000000	0
R3	00	00000000	0
R4	00	00000000	0
R5	00	00000000	0
R6	00	00000000	0
R7	00	00000000	0
R8	00	00000000	0
R9	00	00000000	0
RA	00	00000000	0
RB	00	00000000	0
RC	00	00000000	0
RD	00	00000000	0
RE	00	00000000	0
RF	00	00000000	0

Main Memory (16 x 16 Matrix = 256 Cells):

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
1	21	46	31	FF	21	43	31	FF	21	41	31	FF	21	49	31	FF
2	C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
4	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
5	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
6	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
7	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
8	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
9	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Clear Memory | Load Data | Run | Single Step | Halt | Help

Bonus Task 3: Vole Machine Simulator GUI Snapshots (3 & 4)

C++ Graphical User Interface showing Registers, 16x16 Memory Matrix, Screen Output & Step Controls

3. Single-Step Execution Mode (Inspecting PC=18h, IR=31FF, Screen="FC")

Vole Machine Simulator GUI - CS213 (Section S39_S40) - Dr. Mohammad El-Ramly

Data Input Window (Vole Machine Program):

```
; Print FCAI on Screen (3 RFF)
2146 ; R1 = 'F' (46h)
31FF ; Store R1 to Screen (FF)
2143 ; R1 = 'C' (43h)
31FF ; Store R1 to Screen (FF)
2141 ; R1 = 'A' (41h)
31FF ; Store R1 to Screen (FF)
2149 ; R1 = 'I' (49h)
31FF ; Store R1 to Screen (FF)
C000 ; Halt execution
```

Start Address (Hex Even): 10 | Browse Program File...

Memory-Mapped Screen Output (Address FF):

```
ASCII Output : FC
Hex Stream   : 46 43
Status       : RUNNING (Step-by-Step)
```

CPU & Registers (R0 - RF):

PC : 18 IR : 31FF

Reg	Hex	Binary	Dec
R0	00	00000000	0
R1	43	01000011	67
R2	00	00000000	0
R3	00	00000000	0
R4	00	00000000	0
R5	00	00000000	0
R6	00	00000000	0
R7	00	00000000	0
R8	00	00000000	0
R9	00	00000000	0
RA	00	00000000	0
RB	00	00000000	0
RC	00	00000000	0
RD	00	00000000	0
RE	00	00000000	0
RF	00	00000000	0

Main Memory (16 x 16 Matrix = 256 Cells):

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
1	21	46	31	FF	21	43	31	FF	21	41	31	FF	21	49	31	FF
2	C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
4	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
5	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
6	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
7	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
8	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
9	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	43

Clear Memory | Load Data | Run | Single Step | Halt | Help

4. Full Run Execution Completed (HALT C000 Reached, Screen="FCAI")

Vole Machine Simulator GUI - CS213 (Section S39_S40) - Dr. Mohammad El-Ramly

Data Input Window (Vole Machine Program):

```
; Print FCAI on Screen (3 RFF)
2146 ; R1 = 'F' (46h)
31FF ; Store R1 to Screen (FF)
2143 ; R1 = 'C' (43h)
31FF ; Store R1 to Screen (FF)
2141 ; R1 = 'A' (41h)
31FF ; Store R1 to Screen (FF)
2149 ; R1 = 'I' (49h)
31FF ; Store R1 to Screen (FF)
C000 ; Halt execution
```

Start Address (Hex Even): 10 | Browse Program File...

Memory-Mapped Screen Output (Address FF):

```
ASCII Output : FCAI
Hex Stream   : 46 43 41 49
Status       : HALTED (C000 Reached)
```

CPU & Registers (R0 - RF):

PC : 22 IR : C000

Reg	Hex	Binary	Dec
R0	00	00000000	0
R1	49	01001001	73
R2	00	00000000	0
R3	00	00000000	0
R4	00	00000000	0
R5	00	00000000	0
R6	00	00000000	0
R7	00	00000000	0
R8	00	00000000	0
R9	00	00000000	0
RA	00	00000000	0
RB	00	00000000	0
RC	00	00000000	0
RD	00	00000000	0
RE	00	00000000	0
RF	00	00000000	0

Main Memory (16 x 16 Matrix = 256 Cells):

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
1	21	46	31	FF	21	43	31	FF	21	41	31	FF	21	49	31	FF
2	C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
3	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
4	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
5	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
6	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
7	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
8	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
9	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
A	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
B	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
D	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
E	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
F	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	49

Clear Memory | Load Data | Run | Single Step | Halt | Help